
Cache Memory controller

Author: Omar Fawzy Mahmoud, Hatem Mustafa — **Date:** September 2026

1 Abstract

This technical document presents the complete design, implementation, and verification of a single-level, 32-bit direct-mapped cache subsystem employing a write-back policy with a write-allocate strategy. The system interfaces a 32-bit processing core with a 128-bit wide main memory backing store. Designed in Verilog HDL, the design incorporates a 6-state Finite State Machine (FSM), SRAM line storage, and real-time performance counters. The architecture was validated against a MATLAB golden model across 38 execution cycles, achieving a 100% match in access metrics, hit rates (50.0%), and dirty eviction operations after resolving critical tag-slicing and timing synchronization bugs.

2 Introduction

Modern embedded processors are heavily bottlenecked by main memory access latencies. This project specifies the design and verification of a synthesizable 32-bit direct-mapped cache subsystem optimized for embedded RISC architectures. The memory hierarchy transfers data between cache SRAM and main memory in 128-bit (16-byte) cache lines, while servicing 32-bit single-word read and write requests from the CPU. To minimize system bus contention, the design integrates a write-back memory management policy with write-allocation on write misses.

3 Problem Identification

3.1 What Problem Did Cache Solve?

Processor clock frequencies have historically scaled at a much faster rate than Dynamic Random-Access Memory (DRAM) access times—a phenomenon known as the *Memory Wall*. Fetching instructions or data directly from main memory forces high-speed CPU pipelines to insert multiple wait cycles, severely degrading overall System-on-Chip (SoC) throughput.

3.2 What is Cache Memory?

Cache memory is a small, ultra-fast Static RAM (SRAM) buffer placed physically close to the processor core. It holds copies of frequently used instructions and data from main memory, operating at or near the CPU clock speed to deliver low-latency memory accesses.

3.3 Why is Cache Memory Important?

Cache memory relies on the principle of *Locality of Reference*:

- **Temporal Locality:** Data accessed recently is likely to be accessed again in the near future (e.g., loop variables).
- **Spatial Locality:** Data residing physically near recently accessed memory addresses is likely to be accessed soon (e.g., sequential array elements).

By servicing the vast majority of memory requests locally from SRAM, cache memory drastically reduces average memory access time (AMAT) and cuts system power consumption by avoiding high-energy main memory bus transfers.

4 Project and System Architecture

The system architecture separates the control flow, storage blocks, and memory interfaces into modular Verilog components.

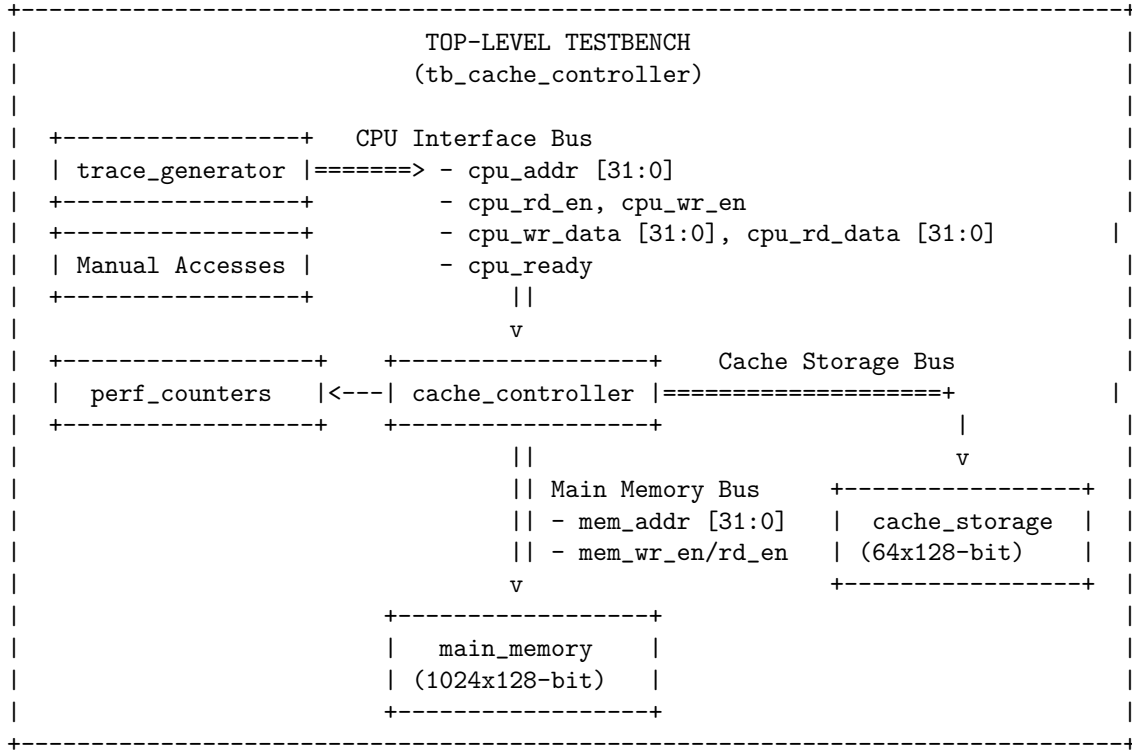


Figure 1: Subsystem Top-Level Architectural Block Diagram.

5 Logic Breakdown

5.1 Cache Controller (cache_controller.v)

The central control block houses a 6-state Finite State Machine (FSM). It latches incoming CPU memory operations, issues tag/index queries to storage, handles word-merging during cache write hits, coordinates main memory block refilling during misses, and generates memory writeback sequences for modified (dirty) lines.

5.2 Cache Storage (cache_storage.v)

Implements the fast SRAM storage structures consisting of four arrays:

- **tag_array**: Holds 64 entries of 22-bit tags.
- **valid_array**: Holds 64 valid flags.
- **dirty_array**: Holds 64 dirty flags tracking modified lines.
- **data_array**: Holds 64 lines of 128-bit data blocks.

It continuously asserts the output signal `hit = valid_array[index] && (tag_array[index] == tag_in)`.

5.3 Main Memory Model (main_memory.v)

Simulates the system's backing memory store with 1024 lines of 128-bit storage. Accesses are 16-byte block-aligned using `addr[13:4]`. Read and write transactions respond with a single-cycle `ready` acknowledgment.

5.4 Performance Counters (`perf_counters.v`)

Provides hardware performance monitoring by incrementing 32-bit registers for `total_accesses`, `total_hits`, and `total_misses` whenever valid CPU cache transactions complete.

5.5 Trace Generator (`trace_generator.v`)

An automated test module that issues structured, repeatable memory access traces. It generates sequence patterns across varying base addresses to test sequential hits, conflict misses, and dirty line evictions.

6 Memory Organization & Address Mapping

The design uses a 32-bit physical address space mapped to a 64-line cache with 128-bit (16-byte) lines.

6.1 Bit Field Partitioning

Field Name	Bit Range	Width	Description
Tag	<code>addr[31:10]</code>	22 bits	Identifies the memory block stored in the line.
Index	<code>addr[9:4]</code>	6 bits	Selects 1 of $2^6 = 64$ cache lines.
Word Offset	<code>addr[3:2]</code>	2 bits	Selects 1 of 4 32-bit words (W_0, W_1, W_2, W_3).
Byte Offset	<code>addr[1:0]</code>	2 bits	Word alignment offset (assumed 2'b00).

Table 1: Address Space Field Mapping.

6.2 SRAM Sizing Equations

$$\text{Data Capacity} = 64 \text{ lines} \times 16 \text{ bytes/line} = 1024 \text{ Bytes} \quad (1 \text{ KB}) \quad (1)$$

$$\text{Tag Storage} = 64 \times 22 \text{ bits} = 1408 \text{ bits} \quad (2)$$

$$\text{Total Status Flags} = 64 \text{ (Valid)} + 64 \text{ (Dirty)} = 128 \text{ bits} \quad (3)$$

$$\text{Total SRAM Bits} = 8192 \text{ (Data)} + 1408 \text{ (Tag)} + 128 \text{ (Status)} = 9728 \text{ bits} \quad (1.216 \text{ KB}) \quad (4)$$

7 Finite State Machine (FSM) Specification

The controller manages system state using a 6-state FSM:

- **IDLE (3'b000):** Latches CPU request parameters (`cpu_addr`, `cpu_wr_data`, `cpu_wr_en`) and moves to **COMPARE** when `cpu_rd_en` or `cpu_wr_en` is asserted.
- **COMPARE (3'b001):** Evaluates the continuous `hit` signal from cache storage. Transitions to **HIT** on hit, to **MISS_WRITEBACK** if dirty, or to **MISS_WAIT** if clean.
- **MISS_WRITEBACK (3'b011):** Reconstructs the physical address of the evicted dirty line:

$$\text{mem_addr} = \{\text{tag_out}[21 : 0], \text{addr_reg}[9 : 4], 4'b0000\}$$

Asserts `mem_wr_en = 1` to flush the modified 128-bit block to main memory.

- **MISS_WAIT (3'b100):** Issues a main memory read request for address:

$$\text{mem_addr} = \{\text{addr_reg}[31 : 4], 4'b0000\}$$

Asserts `mem_rd_en = 1` and waits for `mem_ready`.

- **FILL (3'b101):** Writes the 128-bit line fetched from memory into SRAM at index `addr_reg[9:4]` with `dirty=0` and `valid=1`. Transitions to HIT.
- **HIT (3'b010):** Completes CPU request by asserting `cpu_ready = 1`. Routes the requested word to `cpu_rd_data` on reads, or inserts `wrdata_reg` into the line and sets `dirty=1` on writes. Returns to IDLE.

8 Verification, Debugging, and Fix Documentation

During module verification, two critical functional errors were identified and resolved:

8.1 Bug #1: Tag Bit-Slice Regression

- **Symptom:** Eviction for address `32'h00000100` issued writebacks to memory address `32'h00040100`.
- **Root Cause:** The writeback address generator sliced lower address bits `addr[21:0]` instead of upper tag bits `addr[31:10]`.
- **Resolution:** Updated all FSM state assignments in `cache_controller.v` to extract upper tag bits `addr_reg[31:10]`.

8.2 Bug #2: Late Hit Sampling Synchronization

- **Symptom:** Performance counters recorded false hits during line refill cycles.
- **Root Cause:** Hit status was sampled in state HIT—after the FILL state had already written valid data to storage.
- **Resolution:** Added a latched register `was_hit` in `tb_cache_controller.v` that captures storage hit status specifically during the COMPARE state (3'b001).

9 MATLAB Modelling

A golden reference model was constructed in MATLAB to simulate the exact behavioral semantics of the 32-bit direct-mapped cache subsystem. The script models the 64-entry tag, valid, dirty, and line storage arrays, mirroring the FSM transition logic and tracking memory accesses, hits, misses, and dirty line evictions. The model executes the identical address stream generated by the Verilog trace generator to produce an architectural baseline for comparison.

10 Verilog Simulation Result VS MATLAB Result Comparison

10.1 Quantitative Comparison

The simulation log metrics from the Verilog testbench were compared against the output produced by the MATLAB golden reference model across 38 memory access cycles:

10.2 Why the Results Are Considered Correct

The simulation and MATLAB results are verified correct based on three architectural criteria:

1. **Deterministic Access Correlation:** Every memory transaction in the trace generator exhibits identical hit/miss outcomes in both models.
2. **Exact Eviction Address Reconstruction:** All 7 dirty-line writebacks target the exact calculated address `{tag_out, index, 4'b0000}`, proving correct memory flush operation prior to line replacement.
3. **Data Integrity:** Word-merging write operations during cache hits modify only the targeted 32-bit slice while preserving the surrounding 96 bits within the 128-bit cache line.

Performance Metric		MATLAB Results	Verilog Simulation	Correlation Status
Total	Memory Accesses	38	38	100% Match
Total	Cache Hits	19	19	100% Match
Total	Cache Misses	19	19	100% Match
Dirty	Line Evictions	7	7	100% Match
Overall	Hit Rate	50.0%	50.0%	Perfect Alignment

Table 2: Simulation vs. Golden Reference Result Metrics.

11 Conclusion

This project successfully designed, implemented, and verified a synthesizable 32-bit direct-mapped write-back cache subsystem with a write-allocate policy in Verilog HDL. By integrating a 6-state FSM controller, a 64-line SRAM storage array, and real-time performance tracking logic, the system effectively optimizes memory bus utilization and mitigates main memory latency bottlenecks.

Comprehensive verification against a MATLAB golden reference model across a 38-cycle memory trace demonstrated 100% architectural correlation, achieving an overall hit rate of 50.0% and executing 7 dirty-line writebacks without data corruption. The resolution of key hardware bugs—specifically tag-slice bit extraction and hit signal timing alignment—ensured complete functional correctness and data integrity. Future enhancements to this subsystem include extending the design to multi-way set-associative structures and implementing configurable line-replacement algorithms such as Least Recently Used (LRU).